became the official World Wide Web Consortium (W3C) standard by 2019. WASM employs a human readable *.wat* text format language, which gets converted to a binary *.wasm* format that browsers can run on-the-fly.

This allows us to write code in any language such as C, C++, Rust, etc; compile it to WebAssembly and then run in a web browser just like JavaScript.

This is the fundamental technology that makes it possible to write websites in Python.

Emscripten*:* Emscripten, an open source compiler toolchain, enables developers to develop frontends from any programming language.

It compiles any *portable* C/C++ codebase into WebAssembly, using the LLVM (low level virtual machine) compiler. The LLVM project started in 2000 at the University of Illinois at Urbana–Champaign, under the direction of Vikram Adve (IIT Mumbai alumnus) and Chris Lattner.

The Emscripten output can be used to run on the web, in Node.js or in WASM runtimes.

It focuses on speed, size, and the Web platform.

***Pyodide*:** Pyodide is the foundation of PyScript. It is a Python distribution (a CPython version) for the browser and Node.js, based on WebAssembly.

This Mozilla project allows for wide support of Python packages and the ability to run Python in the browser. PyScript uses Pyodide to execute code located in py-script elements directly in the browser.

Using Pyodide, we can install and run Python packages in the browser with micropip. Micropip is used to install the given package and all its dependencies. This only works for packages that are either pure Python or for packages with C extensions that are built in Pyodide.

Many packages with C extensions have also been ported for use with Pyodide. These include many general-purpose packages like RegEx, PyYAML, lxml and Python packages, including NumPy, Pandas, SciPy, MatPlotLib, and scikit-learn.

We can now connect the dots as to how Python and CPython code can be executed from the web. Figure 1 shows the architecture of PyScript.

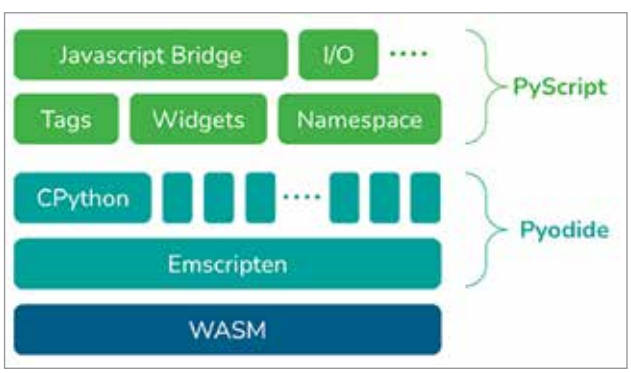

Figure 1: PyScript architecture (Image credits: *https://www.anaconda.com/blog/pyscript-python-in-the-browser*)

Core components of PyScript

Here are a few core components defined in the PyScript documentation. We will explore each with an example. The examples are taken from PyScript's GitHub example repo and real Python link.

PyScript doesn't need to be installed; we only have to import *pyscript.js* and *pyscript.css*. If there is no internet access it can be downloaded and used directly in the HTML files.

Python in the browser

Enable drop-in content, external file hosting, and application hosting without the reliance on server-side configuration. Let's write a simple 'Hello World' Python code in the browser.

Python code is enclosed in the custom py-script tag. Like JavaScript, one can have many py-script tags.

Here is sample code with py-script:

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="https://pyscript.net/alpha/
pyscript.css" />
<script defer src="https://pyscript.net/unstable/pyscript.
js"></script>
</head>
<body>
<py-script >
print("Hello World From Python")
</py-script>

</body>

</html>
```

The output of the code is shown in Figure 2.

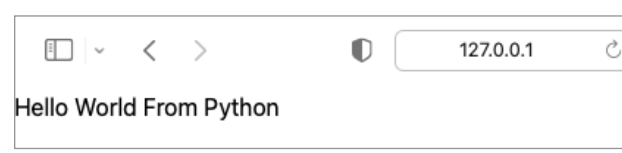

Figure 2: Output of py-script tag

Python ecosystem and environment management

Run many popular packages of Python and the scientific stack (such as NumPy, Pandas, scikit-learn, and more). Allow users to define what packages and files to include for the page code to run.